KLAi

Judging a model: metrics, leakage and imbalance

How do we tell a model that works from one that only looks good?

AI and Data Science Professional | Session 16 of 24 | Module D: Machine learning

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Read a confusion matrix and compute accuracy, precision, recall and F1 from its four counts

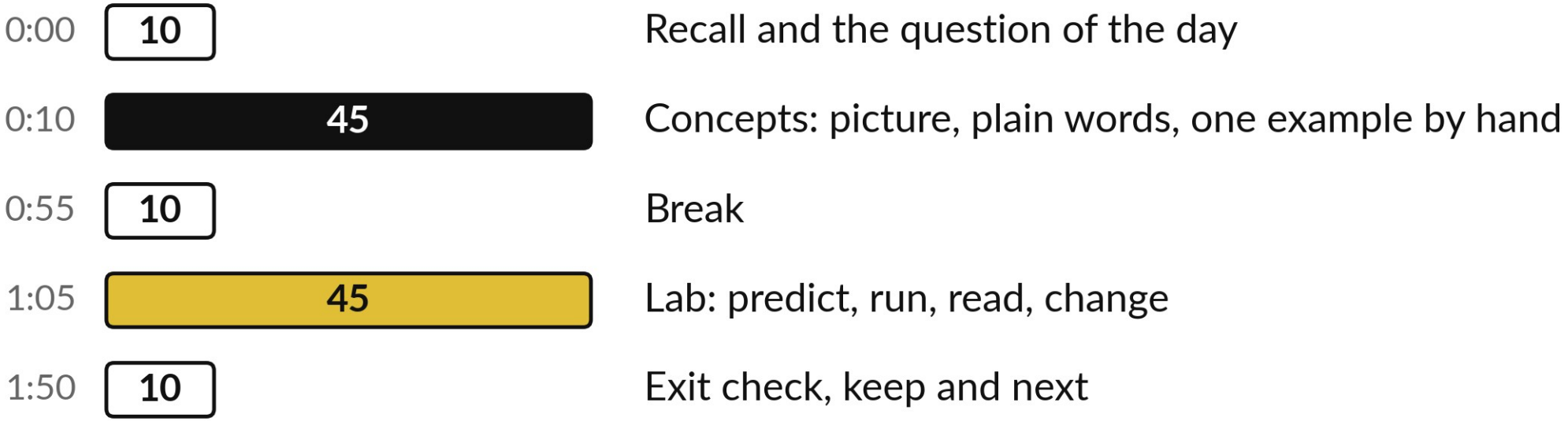


Read an ROC curve and a precision-recall curve, and choose the metric that matches the cost of each mistake



Find a leaked column and test a model with stratified cross-validation

Your two hours: concepts first, then a Python lab in Colab



Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. A logistic regression gives a loan p = 0.30. What does that number mean?

2. Why is 0.50 rarely the right decision threshold?

3. What does one-hot encoding do?



Answer aloud first. Then we check together.

Answers to the three questions from Session 15

	Question	Short answer
	**A logistic regression gives a loan p = 0.30. What does that number mean?**	The model estimates a 30 percent chance that the loan defaults. A decision threshold turns that probability into a yes or no.
	**Why is 0.50 rarely the right decision threshold?**	A missed default and a false alarm cost different amounts. At 0.50 our model flags only 67 of 2,000 loans and misses 200 of the 251 defaults.
	**What does one-hot encoding do?**	It turns a text variable such as occupation into one 0 or 1 indicator column per category, so a model can use it.

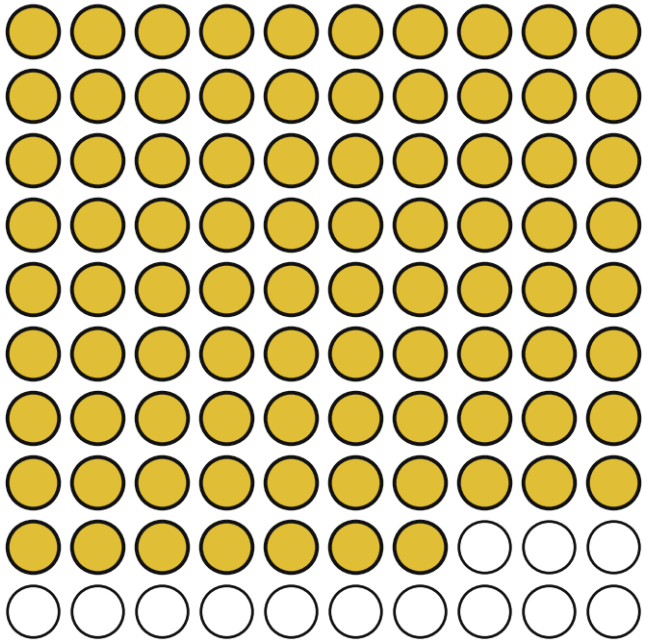
Words we use today

	Word	What it means
	**Confusion matrix (TP, FP, FN, TN)**	A table that counts every kind of right and wrong prediction: true positives, false positives, false negatives and true negatives.
	**Precision, recall and F1**	Precision: of the loans flagged, the share that defaulted. Recall: of the loans that defaulted, the share flagged. F1 combines both.
	**ROC curve and AUC**	The ROC curve plots the true positive rate (recall, sensitivity) against the false positive rate at every threshold. AUC, the area under it, is 0.5 for a coin flip and 1.0 for a perfect model.
	**Data leakage (target leakage)**	A feature that contains the answer, or information only known after the outcome. It makes a model look brilliant and fail in real use.
	**Stratified k-fold cross-validation**	Test the model k times, each time on a different fold, with every fold keeping the same share of defaults. Average the scores. Use it to tune before you test.

A baseline model that flags nobody is right 87.45 percent of the time

87.45%

1,749 of the 2,000 test loans were repaid. Saying "no default" for every loan is correct for all of them, and it finds none of the 251 defaults.



About 87 of every 100 test loans are right

The confusion matrix counts every kind of right and wrong prediction

	Predicted negative (repaid)	Predicted positive (defaulted)
Actual negative (repaid)	**1,579** TN: true negative	**170** FP: false positive
Actual positive (defaulted)	**143** FN: false negative	**108** TP: true positive

Logistic regression at threshold 0.20 on the 2,000 test loans.

Rows: what really happened (actual). Columns: what the model said (predicted).

Positive class: default. Negative class: repaid.

Each loan lands in exactly one of the four cells.

Each cell has a name: the second word is the prediction, the first says whether it was right

	Cell	Name	Plain meaning	Count
	**TP**	True positive	Flagged, and it really defaulted: a good catch.	108
	**FP**	False positive	Flagged, but the loan was repaid: a false alarm.	170
	**FN**	False negative	Not flagged, but it defaulted: a missed default.	143
	**TN**	True negative	Not flagged, and it was repaid: rightly cleared.	1,579

In pandas: ((y_pred == 1) & (y_test == 1)).sum() counts the true positives. A type I error is a false positive, a type II error a false negative (Session 12).

Accuracy is the share of right answers, and it can reward doing nothing

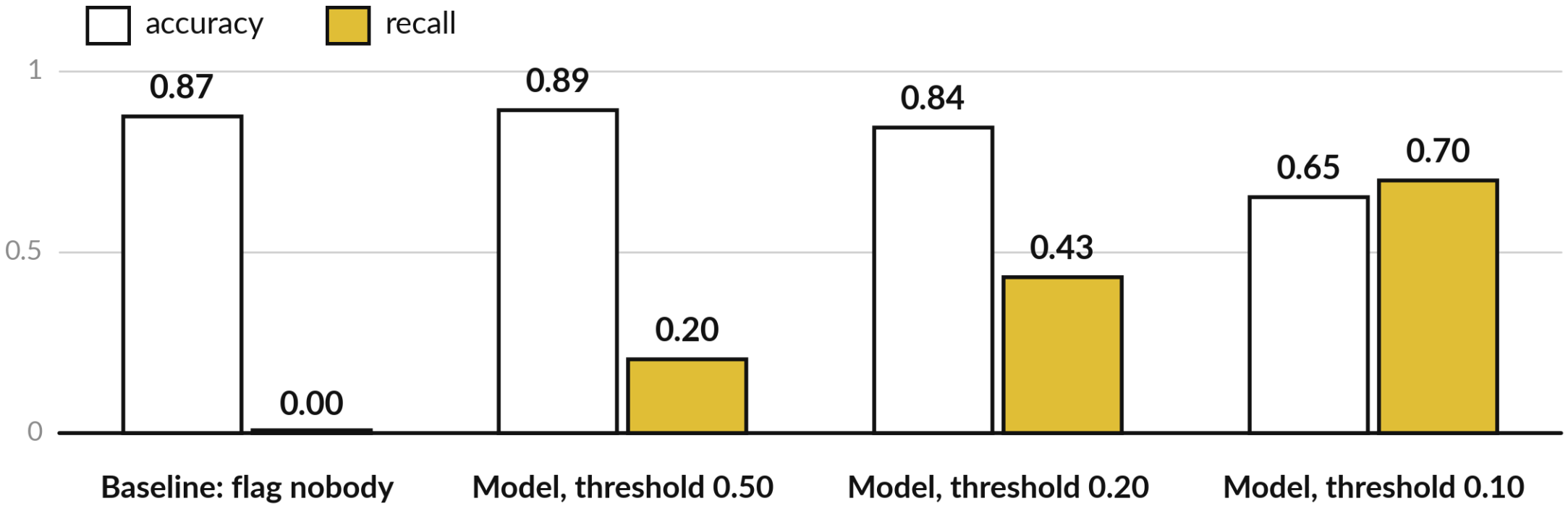
accuracy

$$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

Symbol	What it means	In our data
TP, TN	True positives and true negatives: the right answers	108 and 1,579
FP, FN	False positives and false negatives: the wrong answers	170 and 143
Model at 0.20	(108 + 1,579) / 2,000	0.844
Baseline: flag nobody	(0 + 1,749) / 2,000	0.875
Verdict	The baseline has higher accuracy and catches no default	accuracy paradox

Accuracy paradox: with a rare positive class, accuracy can rise while the model gets less useful.

The accuracy paradox: the model that finds nothing has a high accuracy



Recall: the share of the 251 defaults flagged. Accuracy falls as the model finds more.

Precision and recall each ask a different question about the positive class

precision

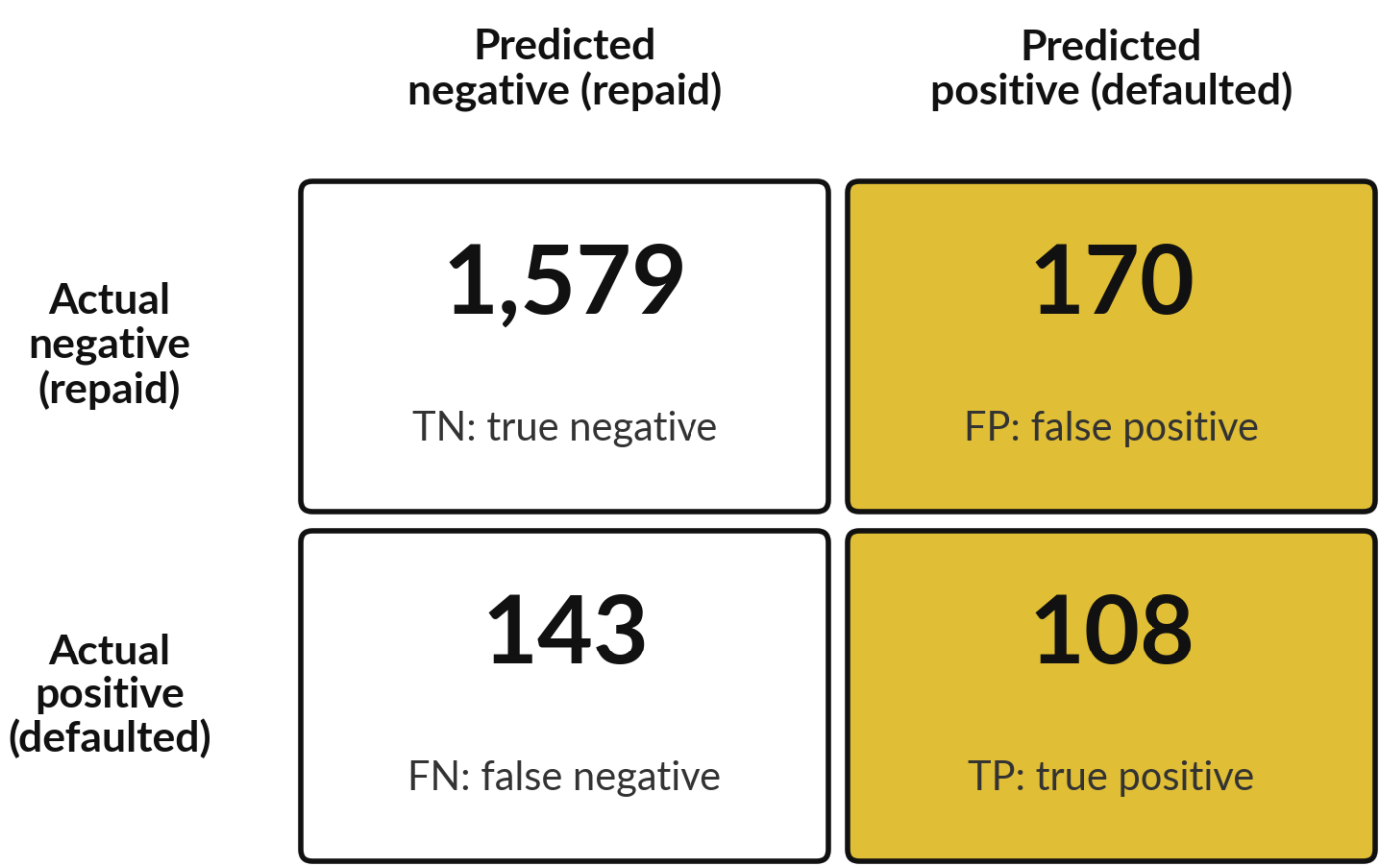
$$\text{precision} = \frac{TP}{TP + FP}$$

recall

$$\text{recall} = \frac{TP}{TP + FN}$$

Symbol	What it means	In our data
Precision	Of the loans flagged, the share that really defaulted	108 / (108 + 170) = 0.388
Recall	Of the loans that defaulted, the share flagged. Also sensitivity or true positive rate	108 / (108 + 143) = 0.430
TP + FP	All loans flagged (the predicted-positive column)	278
TP + FN	All loans that defaulted (the actual-positive row)	251

Precision reads down the column of flagged loans



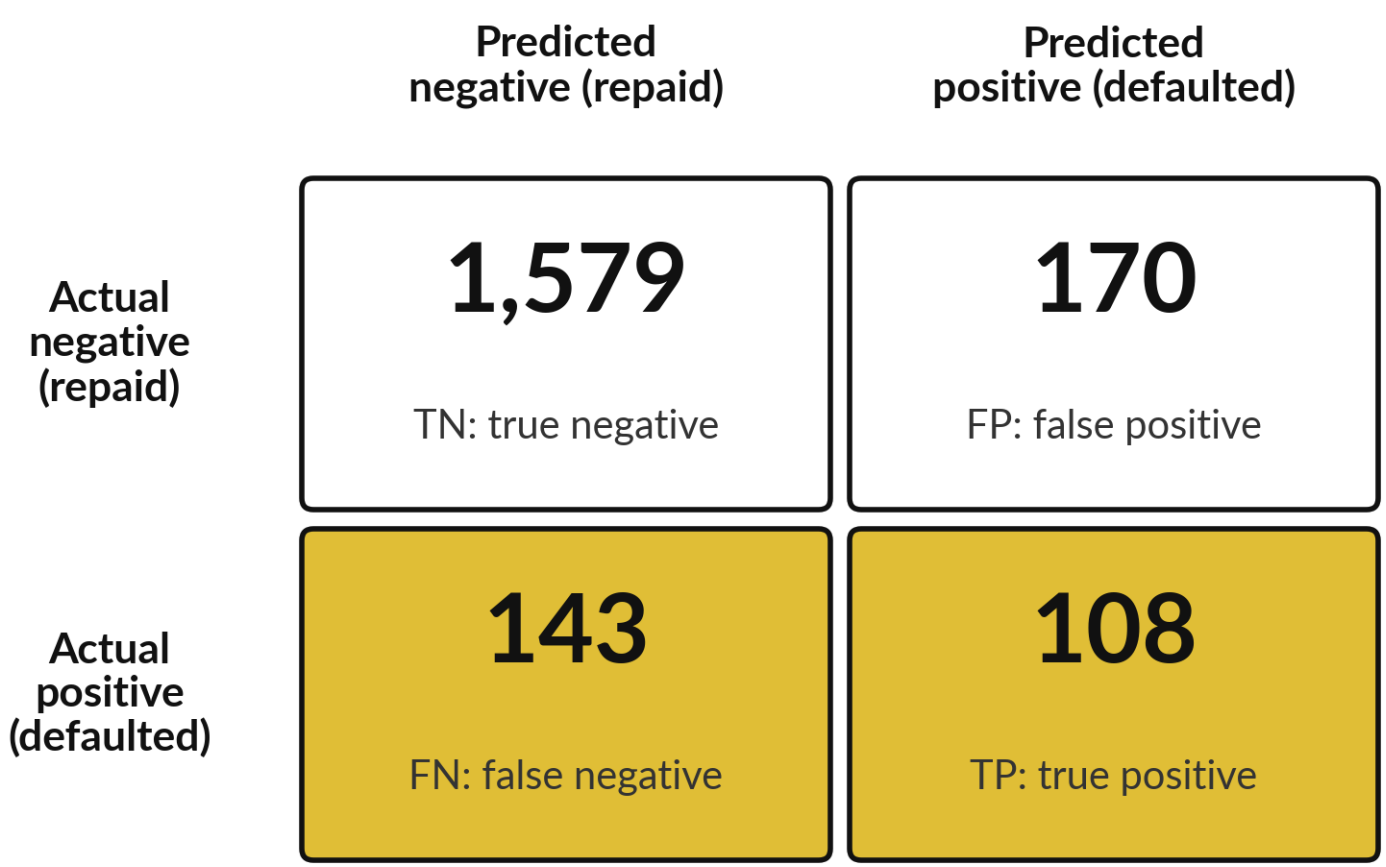
Of 278 loans flagged, 108 defaulted: precision 0.39.

High precision: few false alarms. A flag can be trusted.

Watch it when each flag costs a person's time, such as a manual review.

Low here: about 61 percent of flags are good borrowers.

Recall reads along the row of loans that really defaulted



Of 251 defaults, 108 were flagged: recall 0.43.

High recall: few missed defaults.

Watch it when a miss is expensive, as in lending, disease screening and fraud.

Here the model misses 143 of the 251 defaults.

F1 combines precision and recall in one number, and punishes a big gap between them

F1 score

$$F_1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Symbol	What it means	In our data
Model at 0.20	2 x 0.388 x 0.430 / (0.388 + 0.430)	0.408
Why not the plain mean?	Precision 1.00 and recall 0.01: the mean is 0.505, F1 is	0.020
Name	F1 is the harmonic mean of precision and recall	
Use	One number to compare thresholds or models when both mistakes matter	

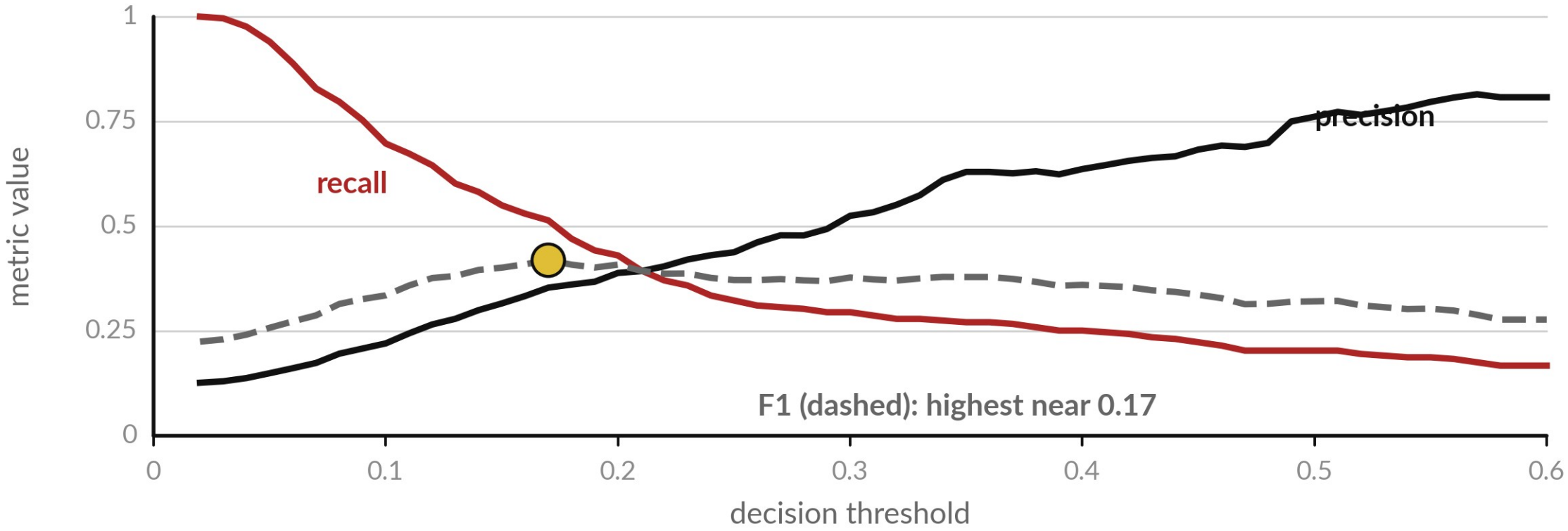
F1 ignores the true negatives. It is 0 when nothing is caught and 1 when every flag is right and every default is found.

Every threshold gives different metrics, and F1 is highest at 0.20 of these five (0.17 if you check every step of 0.01)

	Threshold	Precision	Recall	F1	Accuracy
🔒	**0.50**	0.761	0.203	0.321	0.892
🔒	**0.30**	0.525	0.295	0.378	0.878
💊	**0.20**	0.388	0.430	0.408	0.844
🔔	**0.15**	0.316	0.550	0.401	0.794
🔔	**0.10**	0.220	0.697	0.335	0.652

Baseline for comparison: accuracy 0.875, recall 0. Logistic regression on 2,000 test loans. Gold: F1.

Raising the threshold buys precision and spends recall

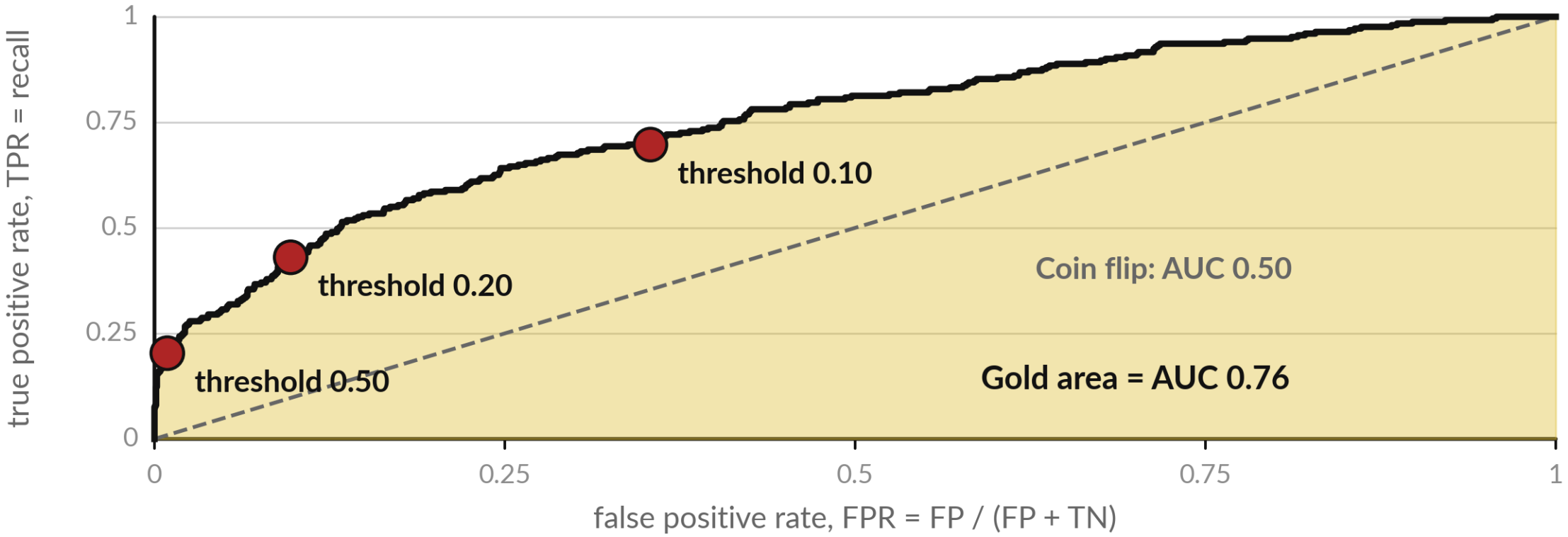


Logistic regression on the 2,000 test loans, thresholds 0.02 to 0.60.

Choose the metric from the cost of the mistake, not from habit

	If the costly mistake is	Watch	Example
	**A missed positive (false negative)**	Recall	A lender who cannot recover a defaulted loan.
	**A false alarm (false positive)**	Precision	A fraud team that can review only 50 cases a day.
	**Both matter and the classes are lopsided**	F1	Comparing two thresholds in one number.
	**You need the quality of the ranking, before choosing a threshold**	AUC	Choosing between two models.
	**The classes are balanced and the costs are equal**	Accuracy	Rare: check the balance first.

The ROC curve plots the true positive rate against the false positive rate at every threshold



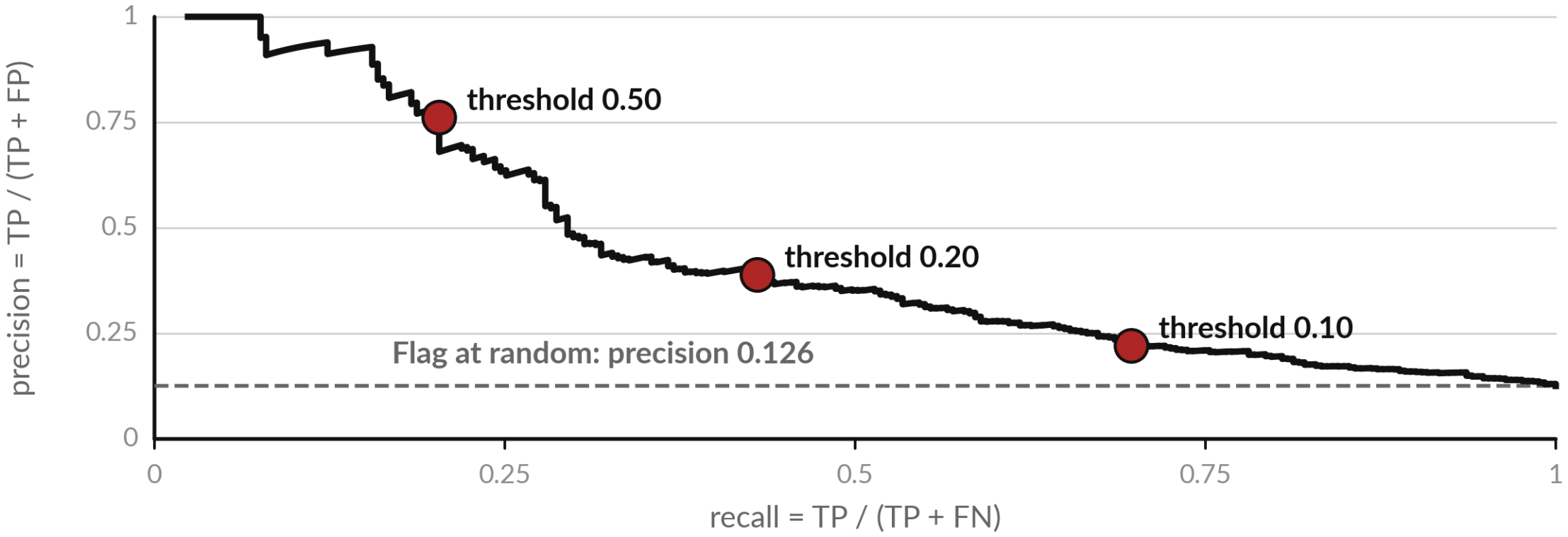
Each dot is one threshold. Lower threshold: more defaults caught, more false alarms.

AUC 0.76: a random defaulter outranks a random repaid loan 76 percent of the time

0.76

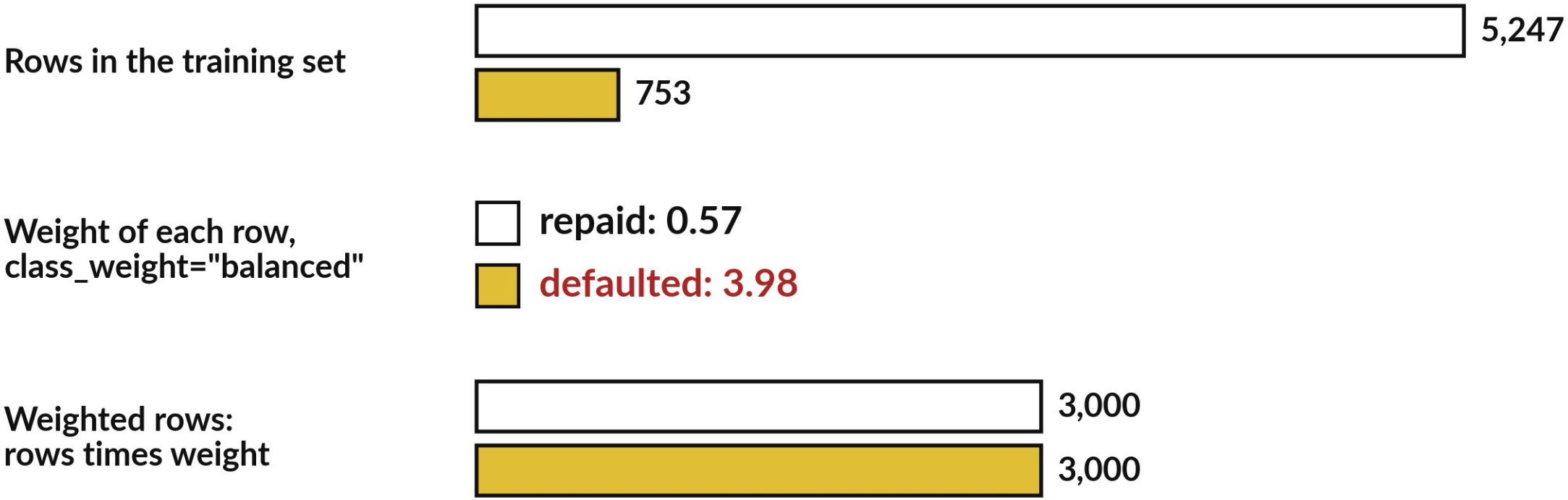
251 defaults and 1,749 repaid loans make 438,999 pairs. In 331,793 the defaulter ranks higher. 0.50 is a coin flip. Gini = 2 x AUC - 1 = 0.51.

The precision-recall curve is the clearer view when the positive class is rare



Each dot is one threshold. A model with no skill sits on the dashed line.

class_weight="balanced" makes each rare default count more when the model learns



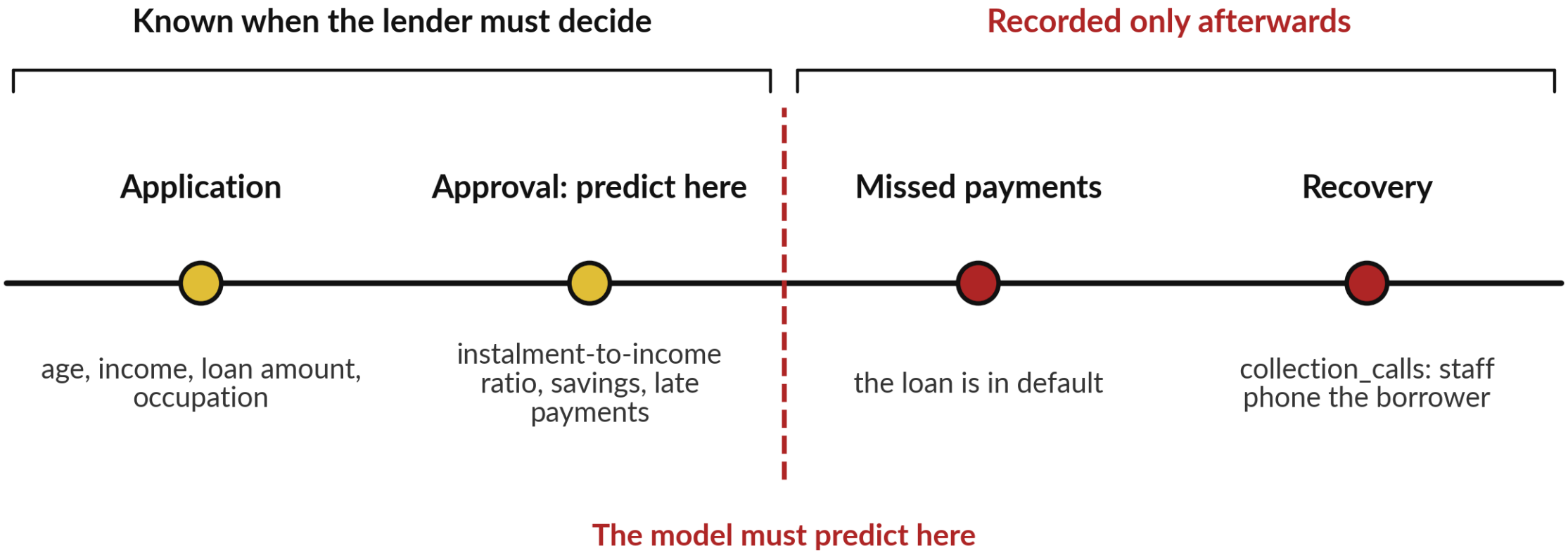
Weight = rows / (2 x rows in class). Each default counts like 7.0 repaid loans.

class_weight shifts the probabilities upward, but the ranking and the AUC hardly change

	Logistic regression	Flagged	Recall	Precision	F1	AUC
🔒	**Plain, threshold 0.50**	67	0.203	0.761	0.321	0.756
🎚	**Balanced, threshold 0.50**	569	0.618	0.272	0.378	0.756
🔔	**Plain, threshold 0.13**	541	0.602	0.279	0.381	0.756

On the 2,000 test loans. Balanced weights at 0.50 behave like the plain model at a threshold near 0.13.

Target leakage: collection_calls is recorded only after the loan has gone bad

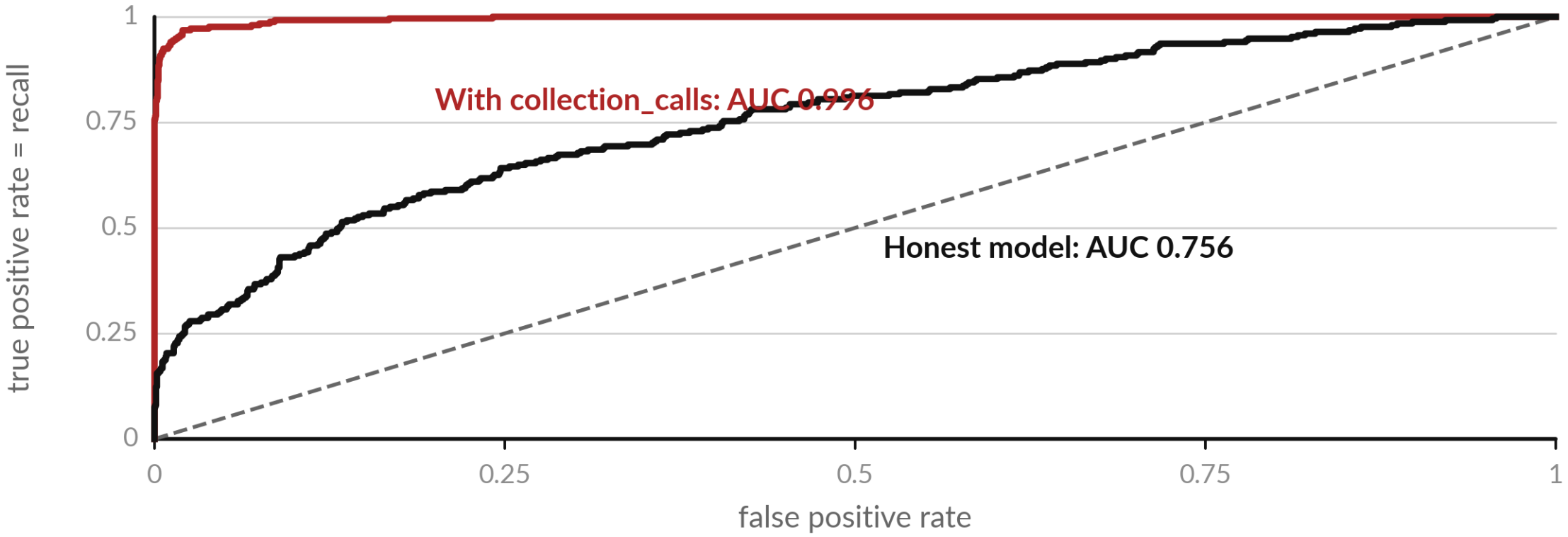


A feature is usable only if it exists at the moment the lender must decide.

The leaked column almost equals the answer: 4.55 calls on average for defaults, 0.24 for repaid loans

Check on the loans file	Result
Average collection_calls, defaulted loans	4.55
Average collection_calls, repaid loans	0.24
Fewest calls on any defaulted loan	1: every default has at least one call
Repaid loans with at least one call	21 percent
Correlation with default, the strongest of all variables	0.87
Next strongest: instalment_to_income	0.37

An AUC of 0.996 should make you suspicious: the leaky model peeks at the answer

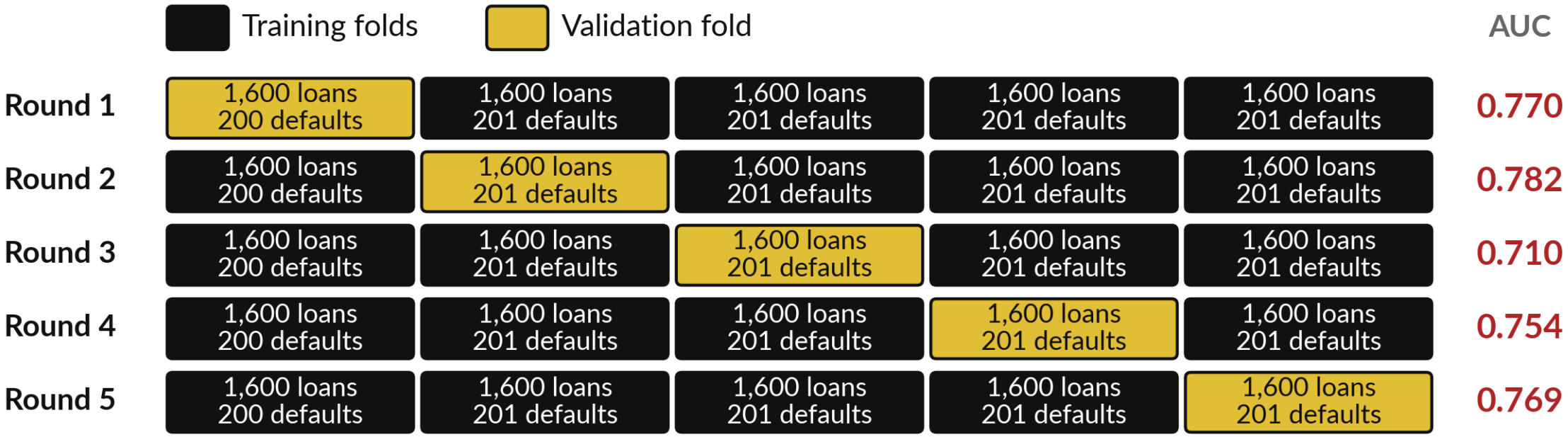


Same loans, same method. At 0.20 the leaky model shows precision 0.88, recall 0.96.

Leakage has several forms, and each has a habit that prevents it

Form of leakage	Example	Prevention
Target leakage: a feature recorded after the outcome	collection_calls in the loans file	Ask when each column is recorded. Drop what is not known at decision time.
Train-test contamination: test rows influence training	Filling missing income with the median of all loans, before the split	Split first. Fit imputers and scalers on the training set only. A scikit-learn Pipeline does this.
Duplicates across the split	The same borrower in the training and test sets	Remove duplicates before splitting, or split by borrower.
Future information (time)	Using next month's repayment to predict this month	Split by date: train on the past, test on the future.

Stratified 5-fold cross-validation tests the model five times, each fold with the same default rate



Mean AUC 0.757, standard deviation 0.028

Five folds of 1,600 loans from all 8,000. Every loan is in the validation fold (gold) once.

Cross-validation agrees: the honest model is about 0.76, and the leaky one is too good to be true

	Model	Round 1	2	3	4	5	Mean	Std
✓	**Honest features**	0.770	0.782	0.710	0.754	0.769	0.757	0.028
✗	**Plus collection_calls**	0.997	0.996	0.997	0.996	0.996	0.996	0.001

AUC on each validation fold of the 8,000 loans. Std: sample standard deviation. Cross-validation cannot detect leakage by itself: the leaked column is in every fold.

Tune before you test: choose settings on a validation set or by cross-validation, then score the test loans once

	Job	Which loans	Example in this course
	**Fit the parameters**	Training loans (6,000)	The coefficients of the logistic regression
	**Tune the settings (decision threshold, tree depth: hyperparameters)**	A validation set, or 5-fold cross-validation inside the training loans (folds of 1,200)	The threshold 0.10 in Session 15. Cross-validated AUC 0.748, spread 0.007
	**Report the final score, once**	Test loans (2,000), never used for tuning	AUC 0.756 and the cost of the chosen threshold

Tuning on the test loans makes the final score optimistic. A validation set is a hold-out part of the training loans; cross-validation repeats that hold-out five times.

Where you will use this at work: report the right metric and check for leaks before you present

	Your job	What you report	What today lets you do
	**Credit risk analyst at a bank, using scikit-learn**	Recall at the chosen decision threshold, and AUC	Show a committee the confusion matrix counts, with more than an accuracy.
	**Fraud analyst at a mobile money provider, using Python**	Precision of the flags the review team receives	Set the threshold so reviewers see a number of cases they can handle.
	**Data scientist reviewing a colleague's model at a telecom**	A leakage check and a cross-validated score	Ask when each feature is recorded before trusting an AUC of 0.99.
	**Monitoring and evaluation officer at an NGO, using Excel and Python**	Recall for a screening model, with the number of false alarms	Explain the trade-off to programme staff in plain counts.

Read the recipe before the lab: seven lines judge a model

```
cm = confusion_matrix(y_test, flag)
print(classification_report(y_test, flag))
auc = roc_auc_score(y_test, y_prob)
w_model = LogisticRegression(max_iter=1000,
    class_weight="balanced")
cv = StratifiedKFold(n_splits=5)
cross_val_score(model, X, y, cv=cv, scoring="roc_auc")
```

In plain words

Line 1: the four counts as a 2 by 2 array. flag is the yes or no at your threshold.

Line 2: precision, recall and F1 for each class in one table.

Line 3: AUC from the probabilities, not from the yes or no flags.

Lines 4 and 5: a model that weights the rare class more.

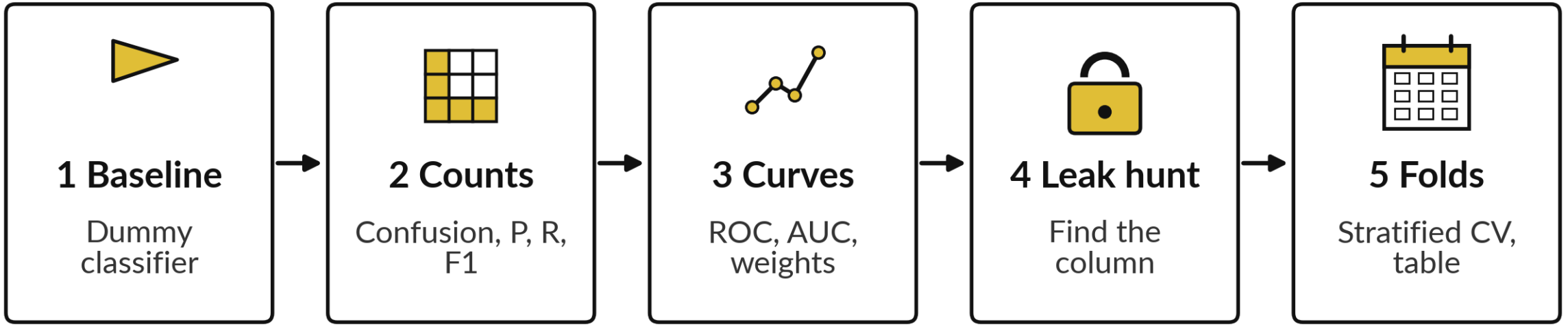
Lines 6 and 7: five stratified folds, one AUC score for each.



Part 2: the Python lab

Close the slides. Open the notebook.

The lab has five steps, and you keep a model comparison table



Compute by hand, then compare with scikit-learn. Step 5 writes model_comparison.csv.

Check your numbers before you move on

	What you compute	You should get
	**Baseline accuracy on the test loans, and its recall**	0.8745 and 0
	**Confusion matrix at 0.20: TP, FP, FN, TN**	about 108, 170, 143, 1,579
	**Precision, recall, F1 at 0.20**	about 0.39, 0.43 and 0.41
	**AUC of the logistic regression on the test loans**	about 0.76
	**Balanced class weight at 0.50: flagged loans and recall**	about 569 and 0.62
	**Column with the strongest correlation with default**	collection_calls, about 0.87
	**Mean 5-fold AUC on all 8,000 loans, honest and leaky**	about 0.757 and 0.996

scikit-learn may differ by a few loans or a unit in the third decimal. If a number is far away, find the step that went wrong.

Five likely mistakes in the lab and how to fix them

	What you see	What it means	What to do
✗	**AUC is about 0.6 (0.60 for flags at 0.50) instead of 0.76**	You passed yes or no flags, not probabilities.	Pass y_prob, the probability column, to roc_auc_score.
✗	**Confusion matrix is the wrong way round**	scikit-learn puts the negative class first.	Read it as TN and FP on top, FN and TP below.
✗	**Precision gives a zero division warning**	Nothing was flagged, so TP + FP is 0.	Lower the threshold, or pass zero_division=0.
✗	**Cross-validation AUC is 0.99**	collection_calls is still in X.	Drop it from the honest features.
✓	**Fold scores differ a lot from the single split**	That is normal variation.	Report the mean and the standard deviation.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**A model has 95 percent accuracy on a file where 5 percent of cases are positive. Is it good?**	Not necessarily. Saying negative for every case also gives 95 percent. Check recall and precision for the positive class.
	**Precision is 0.40 and recall is 0.60. Say each in a sentence about flagged loans.**	Of the loans flagged, 40 percent defaulted. Of the loans that defaulted, 60 percent were flagged.
	**A feature gives an AUC of 0.99. What is the first question to ask?**	When is this feature recorded? If it is only known after the outcome, it is leakage and must be dropped.

Save your notebook and your model comparison table

What you keep from today

Your notebook and a file model_comparison.csv with the evaluation metrics of each model you tested, plus one sentence on the leaked column and what it did to the AUC.

Our question today

How do we tell a model that works from one that only looks good?

Next session

Session 17: Forests, boosting and overfitting. We build stronger models and learn to stop fooling ourselves with them.

Take away

Your lab files

Notebook, comparison

The habit

Never trust one number

Next session

Forests and boosting